

M07 / LEARNER BOOK

PostgreSQL and data modeling

Standalone Zero-to-Job-Ready Data Engineer 2026

13 controlled lessons

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

START HERE | How to use this module 3

DE07-01 | Verify PostgreSQL connection and database/schema context 4

DE07-02 | Tables, data types and explicit DDL 8

DE07-03 | Primary keys, foreign keys and constraints 11

DE07-04 | Transactions, DML and UPSERT 14

DE07-05 | Indexes and EXPLAIN evidence 17

DE07-06 | Operational vs analytical modeling and normalization 20

DE07-07 | Facts, dimensions and star-schema grain 23

DE07-08 | Surrogate keys and referential integrity 27

DE07-09 | Slowly Changing Dimension Type 2 history 30

DE07-10 | Build and validate a small analytical database 34

DE07-11 | Bulk loading with COPY / staging boundaries 38

DE07-12 | Schemas, roles and least-privilege boundaries 43

DE07-13 | Schema change, backup/restore and handoff validation 46

START HERE

How to use this module

Open the matching practice folder from the course root: 05_M07/practice. Work on a copy. Commands printed in a lesson use that module as the working folder unless the lesson names a different location.

Study in lesson order. After the last lesson, use Quick Revision, take the Module Revision Test, then open its separate Protected Answers. Gates are additional assessments at the boundaries shown in the course guide.

From database context and constraints to star schemas, history, staging, privileges and recovery. Build slowly enough to prove every important boundary.

M07 is where SQL stops being only a language you use and becomes a system you are responsible for. In M02 and M03, a table already existed and your job was to query it correctly. In this module you decide what tables should exist, what one row means, which values are allowed, how relationships are protected, how history is preserved, how bulk files cross into governed storage, who is permitted to read or write, and whether another machine can recover the database after you are gone.

The course stays deliberately small enough to check by hand. Our source domain has customers, products, orders and order lines. One customer changes region over time so you can see why current-state data and historical reporting are different questions. Every important transformation has at least two controls - normally a row/key control and a money/value control - because one correct total can hide a missing row while one correct row count can hide wrong values.

Use the visual source when it genuinely helps you picture the idea; use the current PostgreSQL documentation for version-sensitive behavior. Then close outside help before the independent proof. Run DDL, role and backup commands only in a learner-owned local PostgreSQL instance. When native PostgreSQL is unavailable, preserve the task as pending rather than simulating a successful database result.

DE07-01

Verify PostgreSQL connection and database/schema context

Watch first

[1. freeCodeCamp - Learn PostgreSQL Tutorial - Full Course for Beginners](#)

30:15-41:37 - create/connect databases and destructive-command caution

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Watch / current reference

Visual/source: freeCodeCamp - Learn PostgreSQL Tutorial - Full Course for Beginners URL: <https://www.youtube.com/watch?v=qw--VYLpxG4> Assigned scope: 30:15-41:37 - create/connect databases and destructive-command caution Current reference: PostgreSQL 18 - session/system information functions Reference URL: <https://www.postgresql.org/docs/18/functions-info.html> Study rule: Watch/read only the assigned idea, return to this book, type the guided work yourself, then close video/review/AI solving before the independent attempt.

Start here: know where your command will land

Before you create, alter or drop anything, answer four questions from the database itself: Which database am I connected to? Which role am I using? Which schema is first in my search path? Which exact object does an unqualified name resolve to?

Picture PostgreSQL as a building. The PostgreSQL server/cluster is the building. A database is a locked office inside it: one normal connection works inside one database. A schema is a labelled cabinet inside that office. Two cabinets can both contain a file called orders; the full names sales.orders and staging.orders keep them distinct.

search_path is a convenience rule that says which cabinets PostgreSQL looks in first when you write only orders. Convenience becomes risk when two schemas contain the same object name.

Follow the connection proof with me

Run this only in your learner-owned database:

```
SELECT current_database() AS database_name,  
       current_user       AS login_role,  
       current_schema()   AS current_schema,  
       current_schemas(true) AS effective_search_path;  
SHOW search_path;  
SELECT version();
```

Do not merely save a screenshot. Read the result. If your intended practice database is standalone_de but current_database() returns postgres, stop. A perfect DROP TABLE against the wrong database is still a serious mistake.

Create a qualified learner object:

```
CREATE SCHEMA IF NOT EXISTS m07_lab;
```

```
CREATE TABLE m07_lab.connection_check (observed_at timestamptz NOT NULL DEFAULT now(),  
                                         note text NOT NULL);
```

Now compare:

```
SELECT to_regclass('connection_check') AS unqualified,  
       to_regclass('m07_lab.connection_check') AS qualified;
```

`to_regclass` is useful while learning because a missing relation returns NULL instead of raising the same error a direct query would.

Make the search-path risk visible

Create another sandbox schema and deliberately reuse the same table name:

```
CREATE SCHEMA IF NOT EXISTS m07_retry;
```

```
CREATE TABLE m07_retry.connection_check(note text NOT NULL);
```

```
SET search_path TO m07_retry, m07_lab, PUBLIC;
```

```
SELECT current_schema();
```

```
SELECT to_regclass('connection_check');
```

Before running the last line, predict the result. Then explicitly query

`m07_lab.connection_check` and `m07_retry.connection_check`. The lesson is not that

`search_path` is bad; it is that important cross-schema pipeline SQL should not accidentally depend on a session setting nobody noticed.

A second beginner trap: client tree versus active session

In pgAdmin you can click one database in the browser tree and still have an older Query Tool tab connected elsewhere. Trust the active session evidence, not your memory of what you clicked. In `psql`, the prompt and `\conninfo` can help, but the SQL checks above are portable proof.

Your independent proof

Open a fresh connection. Capture database, role, current schema and effective search path before DDL. Create `m07_student` plus one qualified table. Close and reopen your client and prove the object persisted in the intended database/schema.

Then create a second schema with the same table name, alter `search_path` temporarily, and write one sentence explaining why `schema.table` removes ambiguity.

Validate and teach it back

A strong answer is not "schema is like database." Explain the hierarchy accurately: a PostgreSQL server can host several databases; a connection is normally to one database; schemas are namespaces inside that database; tables live inside schemas. State one real failure this distinction prevents.

Changed retry

Use two schemas named raw and serving, each with daily_orders. Predict three resolutions under three search paths, then verify. Restore the original search path after the exercise. Save the prediction before the query result.

One more scenario: the query is correct but the object is wrong

Imagine a pipeline creates daily_orders in schema staging for raw ingestion and another daily_orders in serving for a cleaned consumer table. A developer opens a new session whose search path starts with serving and runs TRUNCATE daily_orders while believing they are clearing staging. PostgreSQL did exactly what the session requested. The failure was context discipline.

A defensive workflow before any destructive DDL/DML is: print the current database; print the current role; qualify the target schema; count or inspect the target object; then change it. For example:

```
SELECT current_database(), current_user;
SELECT count(*) FROM staging.daily_orders;
-- only then, if the learner exercise explicitly requires it:
TRUNCATE staging.daily_orders;
```

The point is not to add ceremony to every SELECT. The point is to slow down before operations whose blast radius is large.

Also remember that temporary tables can temporarily change name resolution because PostgreSQL places a temporary schema early in effective lookup. You do not need to master temporary-schema internals here; simply notice that an unqualified object name is a request for PostgreSQL to resolve a name according to session rules. A schema-qualified name tells it exactly which namespace you mean.

Interview transfer: if someone asks why a query works in one session and fails or points somewhere unexpected in another, mention database connection, schema/search path, permissions and object qualification before blaming SQL syntax.

Evidence before you move on

Database/user/schema/search-path capture + same-name schema resolution test. Keep the genuine first attempt/result/error. A correction made while review is open is repair evidence, not independent mastery.

Revision in your own words

Without opening the lesson, explain Verify PostgreSQL connection and database/schema context in 60-120 seconds. State the row/object grain, one realistic failure, one database-enforced rule (if applicable), and one independent validation. Then reconstruct one small example from memory.

Protected changed retry

Use raw/serving schemas with the same table name; predict three search_path resolutions before querying.

DE07-02

Tables, data types and explicit DDL

Watch first

[1. freeCodeCamp - Learn PostgreSQL Tutorial - Full Course for Beginners](#)

41:37-55:55 - CREATE TABLE, columns and constraints

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Watch / current reference

Visual/source: freeCodeCamp - Learn PostgreSQL Tutorial - Full Course for Beginners URL: <https://www.youtube.com/watch?v=qw--VYLpxG4> Assigned scope: 41:37-55:55 - CREATE TABLE, columns and constraints Current reference: PostgreSQL 18 - CREATE TABLE [Reference URL: https://www.postgresql.org/docs/18/sql-createtable.html](https://www.postgresql.org/docs/18/sql-createtable.html) Study rule: Watch/read only the assigned idea, return to this book, type the guided work yourself, then close video/review/AI solving before the independent attempt.

DDL is a contract, not decoration

When you write CREATE TABLE, you are deciding what future writers are allowed to store. The safest type is not the type that accepts the most data; it is the type that expresses the meaning you intend to protect.

Consider four values that all look numeric on screen:

- 0007 is a warehouse code. Adding 2 to it is meaningless, and losing the leading zeros changes identity. Store it as text.
- 7 cartons is a count. Whole-number arithmetic is meaningful. Store it as an integer.
- 125.50 rupees is an exact business amount. Use exact decimal semantics such as numeric(12,2) rather than treating money as approximate binary floating point.
- 2026-04-01 is a calendar date. Store a date, not a string you later slice.

That decision is data modeling already.

Follow one table definition slowly

```
CREATE TABLE m07_student.products (  
  product_sk bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  product_id text NOT NULL UNIQUE,  
  product_name text NOT NULL,  
  category text NOT NULL,  
  unit_price numeric(12,2) NOT NULL CHECK (unit_price >= 0),  
  active boolean NOT NULL DEFAULT true,  
  created_at timestamptz NOT NULL DEFAULT now()  
);
```

Read every line as a promise. `product_sk` is an internal row identity generated by the database. `product_id` remains a separate business identifier. `numeric(12,2)` means at most 12 total decimal digits with 2 after the decimal point. `DEFAULT true` is used when the `INSERT` omits `active`; it does not magically turn an explicitly supplied `NULL` into `true` when the column is also `NOT NULL`.

Date versus timestamp versus timestamptz

Use `date` for a day such as an invoice date when time-of-day is not part of the fact. Use a `timestamp` when you truly have a time. For event instants exchanged across systems, `timestamptz` is usually the safer concept because PostgreSQL stores an instant and displays it in the session timezone. Do not turn this into a timezone deep-dive yet; the habit here is to ask whether the source means a day or an instant.

DDL that runs can still be wrong

This table will run:

```
CREATE TABLE bad_example (  
    order_id integer,  
    order_date text,  
    amount text  
);
```

But it has thrown away three contracts. `order_id` may lose leading-zero identity; text dates can contain tomorrow; text amount cannot be reliably aggregated without parsing. Successful DDL only proves syntax, not design.

Likewise, an aggressive default can invent business data. If a source omits region, `DEFAULT 'East'` may turn "unknown" into a false fact. Defaults are appropriate only when omission genuinely has a defined meaning.

Inspect what PostgreSQL actually created

Do not trust only the SQL file you intended to run. Query the catalog:

```
SELECT column_name, data_type, is_nullable, column_default,  
       numeric_precision, numeric_scale  
FROM information_schema.columns  
WHERE table_schema='m07_student'  
      AND table_name='products'  
ORDER BY ordinal_position;
```

In `psql`, `\d m07_student.products` is also useful, but it is a client meta-command rather than portable SQL.

Your independent design

Create warehouses, shipments, and `shipment_lines`. Before DDL, write the grain of each table in plain English. Preserve a code such as 0007 as text. Use one generated surrogate key but keep a separate unique business key. Choose exact types for quantity, amount, calendar dates and event timestamps.

Insert one valid row and at least two boundary cases: one that should succeed and one that should be rejected by type/constraint rules. Save the observed database behavior.

Changed retry

Design receipt headers and receipt lines. The source calls a receipt 000023. Explain why converting that business key to integer is a semantic change even though PostgreSQL can numerically store 23.

Work through three type decisions instead of memorizing a type list

Case 1 - money. `numeric(12,2)` gives you an exact decimal contract for the course's rupee-like amounts. That does not mean every financial system must use exactly (12,2); choose precision/scale from realistic maximums and smallest units. A value too large for the declared precision should fail rather than silently widening the contract.

Case 2 - identifiers. Suppose `warehouse_code` is 0007. If you store it as integer 7 and later format it back as four digits, you have created a formatting convention that was not the original identity. If the business can ever introduce A007, the numeric model breaks completely. Store identifiers according to identity semantics, not arithmetic appearance.

Case 3 - missing versus default. If `active` is omitted and the contract truly means new products start active, `DEFAULT true` is sensible. If `region` is absent in the source, defaulting it to East is inventing a fact. Missingness should remain visible until a business rule justifies a value.

When you review DDL, ask four questions per column: What does this value mean? What operations are valid? Can it be absent? Who is allowed to invent a default? That checklist is more transferable than memorizing PostgreSQL type names.

Evidence before you move on

Typed DDL with written grain and catalog inspection, including one meaningful boundary test. Keep the genuine first attempt/result/error. A correction made while review is open is repair evidence, not independent mastery.

Revision in your own words

Without opening the lesson, explain Tables, data types and explicit DDL in 60-120 seconds. State the row/object grain, one realistic failure, one database-enforced rule (if applicable), and one independent validation. Then reconstruct one small example from memory.

Protected changed retry

Design receipt header/line DDL with leading-zero business ID, exact money, date and one generated surrogate.

DE07-03

Primary keys, foreign keys and constraints

Watch first

[1. freeCodeCamp - Learn PostgreSQL Tutorial - Full Course for Beginners](#)

2:29:24-2:49:15 + 3:16:41-3:25:04 - keys, UNIQUE/CHECK and foreign keys

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Watch / current reference

Visual/source: freeCodeCamp - Learn PostgreSQL Tutorial - Full Course for Beginners URL: <https://www.youtube.com/watch?v=qw--VYLpxG4> Assigned scope: 2:29:24-2:49:15 + 3:16:41-3:25:04 - keys, UNIQUE/CHECK and foreign keys Current reference: PostgreSQL 18 - CREATE TABLE constraints Reference URL: <https://www.postgresql.org/docs/18/sql-createtable.html> Study rule: Watch/read only the assigned idea, return to this book, type the guided work yourself, then close video/review/AI solving before the independent attempt.

Constraints are database-level guardrails

Application code can validate input, but a database may receive writes from Python, SQL clients, ETL jobs, admin scripts and future services you have never seen. A well-chosen constraint protects the invariant no matter which writer arrives.

Start with grain. One `order_items` row represents one line inside one order. Therefore `line_no=1` is allowed in thousands of orders; it cannot be globally unique. The row identity is the

pair (`order_id`, `line_no`).

```
CREATE TABLE m07_source.order_items (
  order_id text NOT NULL
    REFERENCES m07_source.orders(order_id),
  line_no integer NOT NULL CHECK (line_no > 0),
  product_id text NOT NULL
    REFERENCES m07_source.products(product_id),
  quantity integer NOT NULL CHECK (quantity > 0),
  unit_price numeric(12,2) NOT NULL CHECK (unit_price >= 0),
  CONSTRAINT pk_order_items PRIMARY KEY(order_id, line_no)
);
```

Naming important constraints makes production errors easier to interpret.

What each rule proves - and what it does not

PRIMARY KEY: this row identity is unique and non-null. It does not prove other business fields are correct.

FOREIGN KEY: a referenced parent key exists. It does not prove the parent is active, belongs to the right company, or is the historically correct version for the event date.

UNIQUE: a value or tuple may occur only once under PostgreSQL's uniqueness semantics. It is useful for business keys, but an SCD2 table intentionally needs several rows for one business customer, so `UNIQUE(customer_id)` would be the wrong rule there.

NOT NULL: absence is forbidden. Empty text "" is not NULL, so required meaningful text may also need a CHECK such as `btrim(name) <> ''`.

CHECK: a row-level expression must not be false. One subtle point: a CHECK expression evaluating to NULL does not reject the row. If you need a value to exist and meet a range, combine NOT NULL with CHECK.

Prove failures inside a safe transaction

Run one test at a time in your sandbox:

```
BEGIN;  
INSERT INTO m07_source.order_items  
(order_id,line_no,product_id,quantity,unit_price)  
VALUES ('0101',99,'P01',0,120);  
ROLLBACK;
```

The quantity CHECK should reject it. In PostgreSQL, once a statement errors inside an explicit transaction, the transaction is typically marked aborted until ROLLBACK (or rollback to a savepoint). Observe this rather than clicking randomly until the error disappears.

Try four deliberate failures: duplicate composite key, missing parent order, zero quantity and blank required text in a table that forbids it. Preserve the error messages. Those errors are successful test evidence.

Do not ask constraints to solve the whole business

Imagine an order references customer C001. The FK can prove C001 exists. It cannot prove the customer's SCD2 region version was valid on the order date. That is a temporal modeling rule you will implement later.

Another example: `quantity > 0` is easy to enforce per row. "Daily shipped quantity may not exceed today's warehouse capacity" depends on multiple rows and changing external state; a simple CHECK is not the right tool.

Your independent proof

Add named constraints to your shipment model. For every constraint, create a two-column note:

1. one bad row this rule catches;
2. one realistic bad state it cannot catch.

That second column is important. It stops you from confusing "database has constraints" with "data is fully correct."

Changed retry

Build receipt lines with primary key (receipt_id,line_no). Deliberately try PRIMARY KEY(line_no) in a disposable version and demonstrate why line 1 of receipt R002 collides with line 1 of R001 even though both are legitimate business rows.

Constraint design also includes deletion behavior

Foreign keys can define actions such as ON DELETE RESTRICT/NO ACTION, CASCADE, SET NULL and others. Do not choose CASCADE because it makes test cleanup easy. If deleting one customer should not silently erase historical orders, cascading from customer to order is probably a dangerous business rule.

For a learner model, the safest default is to make deletion behavior explicit in your reasoning. Ask: if a parent disappears, should child rows be forbidden, deleted, detached, or retained through a different archival process? The database cannot choose the business meaning for you.

Another subtle rule is uniqueness with NULL. Depending on your model and PostgreSQL options, nullable unique fields require thought: "at most one row with this business value" and "every row must have a business value" are two separate contracts. UNIQUE and NOT NULL often work together.

Finally, test the constraint name in the error message. Named constraints such as ck_order_item_quantity_positive communicate more than autogenerated names when a pipeline fails at 2 AM. Good names become operational documentation.

Mini transfer: write one sentence explaining why FOREIGN KEY(customer_id) is excellent at preventing an order for a nonexistent customer, yet useless for proving that the customer's historical region version matches the order date.

Evidence before you move on

Named PK/FK/UNIQUE/CHECK/NOT NULL failure evidence and constraint-boundary explanation. Keep the genuine first attempt/result/error. A correction made while review is open is repair evidence, not independent mastery.

Revision in your own words

Without opening the lesson, explain Primary keys, foreign keys and constraints in 60-120 seconds. State the row/object grain, one realistic failure, one database-enforced rule (if applicable), and one independent validation. Then reconstruct one small example from memory.

Protected changed retry

Receipt line composite key plus missing-parent/blank-text failure set; state what each rule still cannot prove.

DE07-04

Transactions, DML and UPSERT

Watch first

[1. freeCodeCamp - Learn PostgreSQL Tutorial - Full Course for Beginners](#)

2:54:45-3:16:41 - DML, ON CONFLICT and UPSERT

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Watch / current reference

Visual/source: freeCodeCamp - Learn PostgreSQL Tutorial - Full Course for Beginners URL: <https://www.youtube.com/watch?v=qw--VYLpxG4> Assigned scope: 2:54:45-3:16:41 - DML, ON CONFLICT and UPSERT Current reference: PostgreSQL 18 - INSERT / ON CONFLICT Reference URL: <https://www.postgresql.org/docs/18/sql-insert.html> Study rule: Watch/read only the assigned idea, return to this book, type the guided work yourself, then close video/review/AI solving before the independent attempt.

Transactions protect units of work

A data pipeline often performs several related writes. If statement 1 succeeds, statement 2 fails, and statement 3 never runs, leaving statement 1 permanently committed can create a half-built state. A transaction gives you an all-or-nothing boundary when you use that boundary deliberately.

Think of three observable moments: before, inside, after.

```
SELECT product_id, unit_price
FROM m07_source.products
WHERE product_id='P01';           -- before: 120.00

BEGIN;
UPDATE m07_source.products
SET unit_price=130
WHERE product_id='P01';
SELECT product_id, unit_price
FROM m07_source.products
WHERE product_id='P01';           -- inside: 130.00
ROLLBACK;

SELECT product_id, unit_price
FROM m07_source.products
WHERE product_id='P01';           -- after: 120.00 again
```

The proof is not that your script contains the word ROLLBACK. The proof is the state you observed before/inside/after.

What happens after an error

Inside an explicit PostgreSQL transaction, an error usually aborts the transaction. Later commands are rejected until rollback (or rollback to an earlier SAVEPOINT). Beginners often keep issuing commands and think PostgreSQL is broken. Read the first error, not the five follow-on errors it caused.

A savepoint can isolate a smaller experimental step:

```
BEGIN;
SAVEPOINT before_test;
-- run a risky learner test
ROLLBACK TO SAVEPOINT before_test;
COMMIT;
```

You do not need savepoints for every pipeline, but knowing they exist helps you understand transaction state.

UPSERT: rerun safely only when the conflict key matches the grain

Create a current-state feed:

```
CREATE TABLE m07_lab.product_feed (
  product_id text PRIMARY KEY,
  product_name text NOT NULL,
  unit_price numeric(12,2) NOT NULL
);

INSERT INTO m07_lab.product_feed(product_id,product_name,unit_price)
VALUES ('P01','Notebook',125.00)
ON CONFLICT (product_id) DO UPDATE
SET product_name = EXCLUDED.product_name,
    unit_price = EXCLUDED.unit_price;
```

Run the same statement twice. You should still have one P01 row. The rerun converges because the table's intended grain is one current row per product business ID.

Now ask the question that separates engineering from memorizing syntax: Would product_id still be the right conflict target in a price-history table? No. Overwriting the current row would destroy the historical versions you intended to retain.

Idempotent does not mean harmless

A statement can be repeatable and still repeatedly write the wrong value. Idempotency means repeated execution reaches the same intended state for the same input; correctness still depends on the input contract, grain and conflict rule.

Your independent proof

Build one current-state supplier table. Save the state after:

1. first INSERT;
2. changed UPSERT;
3. exact rerun of the changed UPSERT.

Then create a transaction with two related writes where the second intentionally violates a constraint. Roll back and prove the first write did not remain.

Changed retry

Use a different business key and different changed attribute. Then write two sentences explaining why the same UPSERT pattern is appropriate for current-state supplier contact data but inappropriate for SCD2 supplier-risk history.

Auto-commit can hide the transaction boundary from beginners

GUI clients often make ordinary statements convenient by committing automatically when you are not inside an explicit transaction. That convenience becomes confusing when you expect ROLLBACK to undo something that was already committed in a previous action. For the learner lab, run the whole BEGIN ... ROLLBACK block deliberately and observe the same session.

A production pipeline also needs to decide what belongs in one transaction. Putting an entire multi-hour batch in one giant transaction can have operational costs; committing every individual row can leave partial state. M07 does not prescribe a universal batch size. The competency is to identify the consistency boundary: which writes must succeed/fail together for the database to remain meaningful?

For UPSERT, test more than row count. After an exact rerun, verify the business key remains unique and the stored values are correct. Then run a changed input under the same key and prove only the attributes you intend to update are changed. If the source provides an immutable creation timestamp, blindly assigning every column from EXCLUDED can overwrite something that should never change.

Engineering sentence: ON CONFLICT is not a deduplication strategy by itself. It is a deterministic response to a particular declared uniqueness conflict.

Evidence before you move on

Before/inside/after rollback proof + current-state UPSERT exact-rerun proof. Keep the genuine first attempt/result/error. A correction made while review is open is repair evidence, not independent mastery.

Revision in your own words

Without opening the lesson, explain Transactions, DML and UPSERT in 60-120 seconds. State the row/object grain, one realistic failure, one database-enforced rule (if applicable), and one independent validation. Then reconstruct one small example from memory.

Protected changed retry

Supplier current-state upsert + intentional two-write failure; explain why SCD2 cannot use overwrite semantics.

DE07-05

Indexes and EXPLAIN evidence

Watch first

[1. Learn PostgreSQL - database foundations](#)

Optional visual - no verified bounded range. Review the table/key/query context before the EXPLAIN lab. Keep the dedicated query-optimization conference visual for the execution-plan explanation.

[2. Postgres Conference - Query Optimizations and Where to Find Them: A Beginners Guide](#)

OPTIONAL visual - no verified bounded range. Use only for EXPLAIN/index context; PostgreSQL 18 docs + learner lab are required.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Watch / current reference

Visual/source: Postgres Conference - Query Optimizations and Where to Find Them: A [Beginners Guide](#) URL: <https://postgresconf.org/conferences/SEA2024/program/proposals/query-optimizations-and-where-to-find-them-a-beginners-guide> Assigned scope: Current reference: [PostgreSQL 18 - EXPLAIN Reference URL: https://www.postgresql.org/docs/18/sql-explain.html](https://www.postgresql.org/docs/18/sql-explain.html)

Study rule: Watch/read only the assigned idea, return to this book, type the guided work yourself, then close video/review/AI solving before the independent attempt.

OPTIONAL visual - no verified bounded range; EXPLAIN/index context only.

An index is a trade-off, not a trophy

Indexes can reduce the work needed to locate rows, but they consume storage and must be maintained as rows change. PostgreSQL's planner is allowed to ignore an index when a sequential scan is cheaper. Your goal is not to force the words "Index Scan"; your goal is to explain the query shape and plan evidence honestly.

Read EXPLAIN at beginner depth

Start with:

```
EXPLAIN
SELECT order_id, customer_id, order_date
FROM m07_source.orders
WHERE customer_id='C002';
```

The six-row teaching table will probably prefer Seq Scan. That is sensible: reading six rows can cost less than consulting an index and then fetching table pages.

At this stage, learn four ideas:

- scan type - sequential scan, index scan, bitmap scan, etc.;
- estimated rows - how many rows the planner expects;
- cost - planner cost units used to compare alternatives, not milliseconds;

- filter/index condition - which predicate is applied where.

EXPLAIN does not execute an ordinary SELECT. EXPLAIN ANALYZE does execute the statement and adds actual execution evidence. Do not put ANALYZE in front of a modifying statement just because a tutorial did; in a learner course, keep it to safe read-only statements unless you fully understand the consequences.

Build a candidate index from a query pattern

```
CREATE INDEX idx_orders_customer_date
ON m07_source.orders(customer_id, order_date);
```

The column order expresses a likely access pattern: first locate a customer, optionally narrow dates. It is not automatically equivalent to an index designed primarily for date-only searches.

Make the experiment large enough to mean something

A tiny table cannot teach selectivity well. In a disposable schema, generate a larger fixture:

```
CREATE TABLE m07_lab.order_search AS
SELECT g AS row_id,
       'C' || lpad(((g % 1000)+1)::text,4,'0') AS customer_id,
       DATE '2026-01-01' + (g % 180) AS order_date,
       (g % 5000)::numeric(12,2) AS amount
FROM generate_series(1,50000) AS g;
ANALYZE m07_lab.order_search;
```

First query a selective customer/date range and save the result count. Capture EXPLAIN (ANALYZE, BUFFERS) if safe in your local sandbox. Then add an index and compare the same query and same result count. Performance evidence is invalid if the business query changed.

Now change the distribution: ask for a predicate that returns most rows. Predict that the planner may prefer a sequential scan because the index no longer avoids much work.

Common bad conclusions

"Index exists, therefore query is faster" - unsupported.

"Seq Scan means my index is broken" - false on small/unselective inputs.

"Index every column" - increases write/storage cost and often creates useless structures.

"Lower estimated cost on a 6-row table proves production speedup" - training evidence is too small for that claim.

Your independent proof

Choose one query pattern from your own M07 tables. State its predicate, expected selectivity and candidate index before creating the index. Preserve pre/post plans and query-result controls. Write one downside of the index.

Changed retry

Build a dataset where 95% of rows share one hub_id, then compare a lookup for the dominant hub versus a rare hub. Predict the difference before reading the plans.

Selectivity changes the usefulness of the same index

Suppose your 50,000-row fixture has about 50 rows for one customer. A predicate on that customer is selective: most rows are irrelevant. An index can provide a useful shortcut. Now suppose 47,500 rows have `hub_id='H01'`. Looking up H01 returns almost the whole table; sequential reading may be cheaper.

That is why "column is frequently filtered" is not enough to justify an index. Think about how many rows the predicate returns, how the table is stored, whether ordering matters, and how often writes must maintain the index.

For a composite B-tree index (`customer_id`, `order_date`), a query constraining `customer_id` and then a date range fits the ordering well. A date-only query may not benefit in the same way because the leading key is not constrained. Do not turn this into a rigid rule that PostgreSQL can never use later columns - the planner has more strategies - but use leading-column order as your first design intuition.

When comparing plans, call `ANALYZE table_name;` after a large learner data change so planner statistics are refreshed. Otherwise you may be judging a plan based on stale estimates.

Validation rule: plan evidence is secondary to correctness. Before performance comparison, prove both versions return the same row population and key set.

Evidence before you move on

Pre/post EXPLAIN evidence on a query-shaped index with unchanged result controls. Keep the genuine first attempt/result/error. A correction made while review is open is repair evidence, not independent mastery.

Revision in your own words

Without opening the lesson, explain Indexes and EXPLAIN evidence in 60-120 seconds. State the row/object grain, one realistic failure, one database-enforced rule (if applicable), and one independent validation. Then reconstruct one small example from memory.

Protected changed retry

Create dominant-vs-rare hub distribution and predict plan/index usefulness for each predicate.

DE07-06

Operational vs analytical modeling and normalization

Watch first

[1. Decomplexify - Learn Database Normalization \(1NF, 2NF, 3NF...\)](#)

0:00-20:29 - why normalize, 1NF, 2NF and 3NF

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Watch / current reference

Visual/source: Decomplexify - Learn Database Normalization (1NF, 2NF, 3NF...) URL: https://www.youtube.com/watch?v=GfQaEYEc8_8 Assigned scope: 0:00-20:29 - why normalize, 1NF, 2NF and 3NF Current reference: PostgreSQL 18 - table constraints / relational [structure reference](#) Reference URL: <https://www.postgresql.org/docs/18/ddl-constraints.html> Study rule: Watch/read only the assigned idea, return to this book, type the guided work yourself, then close video/review/AI solving before the independent attempt.

Two workloads, two modeling goals

An operational database supports day-to-day state changes: create an order, update a customer, add a line, enforce references. An analytical database supports questions across many events: sales by region/month/product, customer history, trends and reconciliations.

You are not learning "normalized good, star schema good-er." You are learning that a model is designed for a workload and a truth contract.

Start with a bad flat table

Imagine one row per order line containing: order_id, line_no, order_date, customer_id, customer_name, customer_region, product_id, product_name, quantity, unit_price.

If C001 has 10,000 historical lines, the current customer name and region are copied thousands of times. When the customer changes region, which rows should change? If you update every historical row, you may rewrite history. If you update only some, current-state copies disagree. The flat table has mixed several kinds of truth at line grain.

Normalization as practical dependency thinking

You do not need a theorem-heavy detour. Use these questions:

1NF practical idea: each field holds one value of the intended domain; repeating groups such as product1, product2, product3 become separate rows/related tables.

2NF practical idea: in a table whose key has multiple columns, non-key attributes should describe the whole key. In (order_id, line_no), order_date depends on order_id, not the whole line key; it naturally belongs in the order header.

3NF practical idea: non-key attributes should not depend on another non-key attribute in a way that causes repeated update facts. If `customer_id` -> `customer_region`, repeating region on every current order line creates an update anomaly.

The point is not to chant 1NF/2NF/3NF in interviews. The point is to see why customers -> orders -> `order_items` gives each current operational fact one authoritative home.

Operational current state still does not solve history

Suppose Asha is East in January and moves South on 1 February. The operational customers row may correctly show only current region = South. A January report asking "region at order time" needs historical context. Normalization prevented duplicate current-state copies; it did not automatically create history.

Why analytics deliberately reshapes data

A dimensional star says: choose a fact grain, store measurable events there, and attach descriptive dimensions. This can intentionally duplicate/reshape some descriptive information to make analytical joins consistent and understandable. That is a different goal from OLTP update design.

For this course:

- operational grain: one customer, one order header, one order line;
- analytical fact grain: one completed order line;
- customer dimension: descriptive customer history;
- product dimension: product description;
- fact: event keys, quantity and line amount.

Work a small update anomaly by hand

Take O101 and O102 for C001. If customer region is stored only in the current customer table and changes from East to South, both orders would join to South today. That answers "customer's current region." It does not answer "region when the order occurred." You need to label the business question before choosing the model.

Your independent proof

Start from a flat shipment export with repeated hub/coordinator fields. Draw a normalized operational model and a star model. For each table, write one sentence stating grain. Mark every attribute that can change. For one changing attribute, state whether the business needs only current state or historical attribution.

Changed retry

Use supplier receipts. Find one insertion, update or deletion anomaly in a flat design, normalize it, then design an analytical fact at receipt-line grain. Explain why the two models are not competitors; they serve different responsibilities.

Normalize one small example step by step

Start with a spreadsheet-like row: `order_id=O101`, `line1_product=P01`, `line1_qty=2`, `line2_product=P02`, `line2_qty=3`. The repeating `line1/line2/...` groups make the number of products part of the table structure. Converting them into `order_items(order_id,line_no,product_id,quantity)` gives each line the same shape - the practical 1NF improvement.

Now suppose `order_items` has composite key (`order_id,line_no`) but also stores `order_date`. `order_date` depends only on `order_id`, not the full composite line key. Moving it to `orders` avoids repeating the header fact on every line - the practical 2NF idea.

Finally suppose `orders` stores both `customer_id` and `customer_region`, while `region` is really an attribute of the current customer. Repeating the `region` across orders creates a dependency through another non-key fact. Keeping current customer attributes in `customers` is the practical 3NF direction.

These examples are deliberately simplified. Real normalization can go deeper and business dependencies can be nuanced. The job-ready skill is to see repeated authoritative facts and update anomalies, state the dependency you believe exists, and model deliberately.

Then make the analytical choice separately. A star schema may intentionally place convenient descriptive attributes in dimensions because the analytical workload values consistent slicing and history. That is not "breaking normalization accidentally"; it is a different designed model.

Evidence before you move on

Normalized operational model and analytical star from the same flat domain, with update anomaly/history reasoning. Keep the genuine first attempt/result/error. A correction made while review is open is repair evidence, not independent mastery.

Revision in your own words

Without opening the lesson, explain Operational vs analytical modeling and normalization in 60-120 seconds. State the row/object grain, one realistic failure, one database-enforced rule (if applicable), and one independent validation. Then reconstruct one small example from memory.

Protected changed retry

Supplier receipt flat export -> normalized operational model -> star; identify one update anomaly and one history question.

DE07-07

Facts, dimensions and star-schema grain

Watch first

[1. TechLake - DWH Tutorial 6: Star Schema in Dimensional Modeling](#)

OPTIONAL visual support - no independently verified bounded range. Use only to visualize star schema, facts and dimensions; the learner book + PostgreSQL lab are the required route.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Watch / current reference

Visual/source: TechLake - DWH Tutorial 6: Star Schema in Dimensional Modeling URL:

<https://www.youtube.com/watch?v=vdRMTa6t1dM> Assigned scope: [OPTIONAL visual support](#) - no independently verified bounded range. Use only to visualize star schema, facts and dimensions; the learner book + PostgreSQL lab are the required route. Current reference: Course-owned grain/reconciliation implementation in PostgreSQL Reference URL:

<https://www.postgresql.org/docs/18/tutorial-join.html> Study rule: [Watch/read only the assigned](#) idea, return to this book, type the guided work yourself, then close video/review/AI solving before the independent attempt.

Grain first, tables second

The most expensive dimensional-model mistakes often start before any SQL: nobody agreed what one fact row means. If one fact row is an order line, every key and measure must be interpreted at order-line grain.

Our supplied source has 9 order lines totaling 2,280.00. Order O104 is Cancelled with one 500.00 line. The analytical completed-sales population therefore has:

- 8 fact lines;
- 5 distinct completed orders;
- 1,780.00 line amount.

Those three numbers describe the same filtered population at different summaries. We will use all three because one can catch errors another misses.

Build the star in your head

fact_order_line contains repeated measurable events. dim_customer describes the customer version. dim_product describes the product.

A fact row may contain: order_id, line_no, order_date, customer_sk, product_sk, quantity, line_amount.

order_id can remain as a degenerate dimension-like business reference in the fact because it identifies the transaction without needing a separate order dimension for this tiny model. You do not need the term for the job; you need to understand why the identifier can live there without repeating an order-level measure.

Follow the double-counting trap

O101 has two lines:

- 2 notebooks x 120 = 240
- 3 pens x 20 = 60

Total = 300.

If you store order_total=300 on both line rows, then SUM(order_total) returns 600 for O101. The value is correct per order but placed at the wrong grain.

The same danger applies to one-time shipping fees, order discounts, customer credit limits and percentages. Before adding a measure to a fact table, ask: Is this value true once per fact row? Is it additive across the dimensions I plan to sum over?

Build and reconcile

The supplied 05_build_star_schema.sql loads only Completed orders. Before running it, predict the controls:

Control	Expected
fact lines	8
distinct orders	5
fact amount	1780.00
cancelled amount excluded	500.00

Because C001 changes region historically, the final region amounts should be:

- East = 660.00 (O101 300 + O105 360)
- South = 600.00 (O102)
- West = 520.00 (O103 320 + O106 200)
- North = 0 completed amount.

Check the arithmetic: $660 + 600 + 520 = 1,780$.

If your grand total is correct but East/South are wrong, the problem may be historical dimension attribution rather than fact amount. That is why controls should challenge different failure modes.

Dimension keys must not multiply facts

A dimension join must resolve to exactly one appropriate dimension row per fact. If your SCD2 join matches two overlapping customer versions, one source fact line can become two warehouse rows. A fact total may then double. Row/key uniqueness checks are part of dimensional modeling, not optional QA afterthoughts.

Your independent proof

Design a shipment-line fact with hub, carrier and service dimensions. List:

- fact grain;
- fact primary/business uniqueness rule;
- additive measures;
- one header-level value you refuse to repeat at line grain;
- source-to-fact count and amount controls.

Build a small fixture and reconcile it independently.

Changed retry

Switch to receipt-line grain. Introduce one header-level freight amount and explain three valid possibilities: keep it at header grain in another fact/table; allocate it with a documented rule; or exclude it from the line fact. "Repeat it on every line" is not valid just because the SQL is easy.

Fact rows can carry different kinds of columns

At order-line grain, quantity and line_amount are measures. customer_sk and product_sk are dimension references. order_id is a transaction identifier useful for counting distinct orders or tracing back to source. order_date can be kept directly for this learner model, though larger warehouses often use a date dimension for calendar attributes such as fiscal period, weekday or holiday flags.

The important rule is not "every fact must look exactly like a textbook star." It is that every column has a clear role at the declared grain.

Test additivity with questions. SUM(line_amount) across product, customer and date is valid because each line owns that amount once. AVG(unit_price) is not automatically a business KPI across all dimensions because an average of line prices can answer a different question from weighted average selling price. A customer satisfaction score may be non-additive. Inventory balance can be semi-additive across products/locations but not time. You only need the intuition now: do not sum a column merely because it is numeric.

When you join dimensions for reporting, preserve fact row count unless the business question intentionally filters facts. If a dimension join unexpectedly changes 8 fact rows to 9, investigate cardinality before trusting any aggregate.

Evidence before you move on

Star fact with declared grain, additive-measure reasoning and source-to-fact row/value reconciliation. Keep the genuine first attempt/result/error. A correction made while review is open is repair evidence, not independent mastery.

Revision in your own words

Without opening the lesson, explain Facts, dimensions and star-schema grain in 60-120 seconds. State the row/object grain, one realistic failure, one database-enforced rule (if applicable), and one independent validation. Then reconstruct one small example from memory.

Protected changed retry

Receipt-line fact with one header freight amount; choose a safe modeling strategy and reconcile changed controls.

DE07-08

Surrogate keys and referential integrity

Watch first

[1. freeCodeCamp - Learn PostgreSQL Tutorial - Full Course for Beginners](#)

2:29:24-2:49:15 + 3:16:41-3:25:04 + 3:50:42-3:57:18 for Serial & Sequences

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Watch / current reference

Visual/source: freeCodeCamp - Learn PostgreSQL Tutorial - Full Course for Beginners URL:

<https://www.youtube.com/watch?v=qw--VYLpxG4> Assigned scope: 2:29:24-2:49:15 +

3:16:41-3:25:04 + 3:50:42-3:57:18 for Serial & Sequences Current reference: PostgreSQL 18 - identity columns and constraints Reference URL:

<https://www.postgresql.org/docs/18/ddl-identity-columns.html> Study rule: Watch/read only the

assigned idea, return to this book, type the guided work yourself, then close video/review/AI solving before the independent attempt.

Business identity and warehouse row identity are different jobs

A source may call the business customer C001 for years. A warehouse may need several historical rows for C001. If every historical row used customer_id as the primary key, you could store only one version. A surrogate key gives each warehouse version its own row identity.

Picture two labels on a dimension row:

- customer_id='C001' answers which business customer?
- customer_sk=17 answers which warehouse row/version?

The fact stores the surrogate key because a January event must attach to the January-valid dimension version, not whichever customer row happens to be current today.

Follow the SCD2 key design

```
CREATE TABLE m07_dw.dim_customer (
    customer_sk bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    customer_id text NOT NULL,
    customer_name text NOT NULL,
    region text NOT NULL,
    valid_from date NOT NULL,
    valid_to date NOT NULL,
    is_current boolean NOT NULL,
    CONSTRAINT uq_customer_version UNIQUE(customer_id, valid_from),
    CONSTRAINT ck_customer_window CHECK(valid_from < valid_to)
);
```

```
CREATE UNIQUE INDEX uq_customer_one_current
ON m07_dw.dim_customer(customer_id)
```

```
WHERE is_current;
```

The primary key makes each warehouse row unique. (customer_id, valid_from) stops duplicate versions starting on the same date. The partial unique index allows many old versions but only one row marked current for each business customer.

Why surrogate keys do not solve duplicates by themselves

Imagine your loader inserts the same current customer twice. PostgreSQL can happily generate customer_sk=21 and customer_sk=22. Both surrogate values are unique, so the primary key is satisfied. Business duplication remains unless another business/history rule rejects it.

This is the key interview sentence: surrogate uniqueness is row identity; it is not business-entity uniqueness.

Referential integrity at warehouse grain

fact_order_line.customer_sk can have a foreign key to dim_customer.customer_sk. This

proves the fact points to an existing dimension version. It still does not prove the version was temporally valid for order_date; your as-of load logic/validation proves that.

Late-arriving or unknown members

Some warehouses use a declared Unknown dimension row so facts can load when a descriptive record is missing. That can be a legitimate policy, but it must be explicit. A good unknown-member process preserves the original missing business key, counts the unresolved facts, and has a repair path. Silently mapping every bad customer to a real-looking fake row hides source defects.

Do not copy a magic -1 key pattern merely because tutorials do. If you use a reserved unknown key, document how identity generation and constraints accommodate it.

Your independent proof

Create a carrier dimension with generated carrier_sk and business carrier_code. State which key the fact stores, which uniqueness rule protects the current carrier identity, and how history would change that rule.

Deliberately try to load a duplicate business carrier under a fresh surrogate key and show which additional constraint/process catches it.

Changed retry

Design a product SCD2 dimension. Demonstrate why UNIQUE(product_id) prevents legitimate history while UNIQUE(product_id,valid_from) plus one-current-row control matches the intended versioned grain.

Follow one fact row through the keys

O101 belongs to business customer C001. On 2026-01-05, the valid dimension row for C001 is the East version. Suppose that row's surrogate key is 1. O102 on 2026-02-10 belongs to the South version, perhaps surrogate key 2.

The fact rows therefore retain different customer_sk values even though both source orders contain the same customer_id='C001'. This is exactly what allows a historical report to remain stable after the customer's descriptive attributes change.

If you kept only customer_id in the fact and joined to the current dimension row later, both events could be labelled South. The natural key alone identifies the entity but not the historical version.

Now think about a late-arriving dimension. A fact for customer C999 arrives before descriptive customer data. Three honest options might exist depending on architecture: hold/reject the fact; load it against an explicit unknown/inferred member while preserving C999 for later repair; or load to a staging area until the dimension catches up. What is not honest is silently mapping C999 to an unrelated existing customer just to satisfy the foreign key.

Your evidence should always preserve the business key somewhere so a surrogate-key mapping can be audited back to source identity.

Evidence before you move on

Natural/business key + surrogate key model, duplicate protection and version-key reasoning. Keep the genuine first attempt/result/error. A correction made while review is open is repair evidence, not independent mastery.

Revision in your own words

Without opening the lesson, explain Surrogate keys and referential integrity in 60-120 seconds. State the row/object grain, one realistic failure, one database-enforced rule (if applicable), and one independent validation. Then reconstruct one small example from memory.

Protected changed retry

Product SCD2 dimension where one business product legitimately has several surrogate versions.

DE07-09

Slowly Changing Dimension Type 2 history

Watch first

[1. TechLake - DWH Tutorial 16: Slowly Changing Dimensions and Types](#)

OPTIONAL visual support - no independently verified bounded range. Use only for SCD type concepts; the learner book owns the PostgreSQL SCD2 implementation.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Watch / current reference

Visual/source: TechLake - DWH Tutorial 16: Slowly Changing Dimensions and Types URL: <https://www.youtube.com/watch?v=-XjPvozl-So> Assigned scope: OPTIONAL visual support - no independently verified bounded range. Use only for SCD type concepts; the learner book owns the PostgreSQL SCD2 implementation. Current reference: PostgreSQL 18 - range constraints / exclusion concepts (advanced reference only) Reference URL: <https://www.postgresql.org/docs/18/ddl-constraints.html> Study rule: Watch/read only the assigned idea, return to this book, type the guided work yourself, then close video/review/AI solving before the independent attempt.

History is a time-window problem

If Asha was East in January and South from 1 February, a January order should not move to South when you rerun last month's report. SCD Type 2 keeps separate dimension versions and joins an event to the version valid at event time.

Use half-open intervals:

- East: [2025-01-10, 2026-02-01)
- South: [2026-02-01, 9999-12-31)

The left boundary is inclusive; the right boundary is exclusive. An order exactly on 1 February belongs only to the South row. Adjacent windows can touch without overlapping.

Build one change transactionally

Assume C001 currently has East with valid_to='9999-12-31' and is_current=true. A simplified change on 2026-02-01 is:

```
BEGIN;
```

```
UPDATE m07_dw.dim_customer
SET valid_to = DATE '2026-02-01',
    is_current = false
WHERE customer_id='C001'
    AND is_current=true;
```

```
INSERT INTO m07_dw.dim_customer
```

```
(customer_id,customer_name,region,valid_from,valid_to,is_current)
VALUES
('C001','Asha','South',DATE '2026-02-01',DATE '9999-12-31',true);

COMMIT;
```

The two writes belong together. If the new row fails, you do not want the customer left with no current version.

The as-of fact join

```
JOIN m07_dw.dim_customer dc
  ON dc.customer_id = o.customer_id
 AND o.order_date >= dc.valid_from
 AND o.order_date < dc.valid_to
```

This should map O101 on 2026-01-05 to East and O102 on 2026-02-10 to South.

Joining facts with `dc.is_current=true` instead would answer a different question: "what is the customer's current region now?" Both can be legitimate reports. The defect is using one while claiming the other.

Three invariants you must prove

1. One current row per business key. The partial unique index can enforce this directly.
2. Every version has a valid window. `CHECK(valid_from < valid_to)` prevents zero/negative windows.
3. Windows do not overlap. Basic PK/FK rules do not prove this. In an advanced PostgreSQL design, exclusion constraints/range types can enforce non-overlap; in this beginner module, you must at least validate it explicitly.

A simple overlap check for two rows of the same customer is conceptually: window A starts before B ends and B starts before A ends. Use a self-join or window-function validation to search for such pairs.

Check the history by business result

For C001:

- O101 should report East and 300.00;
- O102 should report South and 600.00.

If both report South, you joined to current state. If either order appears twice, your history windows likely overlap or your join condition is incomplete.

SCD1 versus SCD2 is a business decision

If a typo in customer name should simply be corrected everywhere, SCD1 overwrite may be sufficient. If region/risk/tier at event time matters, SCD2 can be justified. History everywhere is not maturity; it is extra complexity and storage that must answer a real question.

Your independent proof

Create dim_hub where H01 changes region on a known date. Load facts before, on, and after the boundary date. Prove old/new attribution, one current row, valid windows and no overlaps.

Changed retry

Use supplier risk tier Low -> High. Produce two reports: "risk when receipt occurred" and "current supplier risk." Explain why they intentionally use different joins.

Write a non-overlap check you can actually run

After loading dimension history, search for overlapping versions:

```
SELECT a.customer_id,  
       a.customer_sk AS a_sk, a.valid_from AS a_from, a.valid_to AS a_to,  
       b.customer_sk AS b_sk, b.valid_from AS b_from, b.valid_to AS b_to  
FROM m07_dw.dim_customer a  
JOIN m07_dw.dim_customer b  
  ON a.customer_id = b.customer_id  
 AND a.customer_sk < b.customer_sk  
 AND a.valid_from < b.valid_to  
 AND b.valid_from < a.valid_to;
```

For well-formed half-open history, this should return zero rows. The a.customer_sk < b.customer_sk condition prevents comparing each pair twice and avoids comparing a row to itself.

Then check current-row count:

```
SELECT customer_id,  
       count(*) FILTER (WHERE is_current) AS current_rows  
FROM m07_dw.dim_customer  
GROUP BY customer_id  
HAVING count(*) FILTER (WHERE is_current) <> 1;
```

Again, a clean result is zero rows.

Finally, test the boundary date itself. A fact exactly on 2026-02-01 must match South, not both versions. Boundary tests are valuable because many temporal bugs hide at <= versus <.

If your business stores timestamp-effective changes rather than day-effective changes, the same half-open idea applies with timestamp/timestamptz values; do not round them to dates unless the business contract truly works at date grain.

Evidence before you move on

SCD2 windows, one-current-row check, non-overlap validation and as-of fact attribution. Keep the genuine first attempt/result/error. A correction made while review is open is repair evidence, not independent mastery.

Revision in your own words

Without opening the lesson, explain Slowly Changing Dimension Type 2 history in 60-120 seconds. State the row/object grain, one realistic failure, one database-enforced rule (if applicable), and one independent validation. Then reconstruct one small example from memory.

Protected changed retry

Supplier risk Low->High; compare current-risk report with risk-at-receipt report.

DE07-10

Build and validate a small analytical database

Watch first

[1. Learn PostgreSQL - complete beginner course](#)

Optional visual - no verified bounded range. Use this as a SQL and database workflow refresher. COPY and backup/restore commands follow the current PostgreSQL instructions printed in the lesson. For a cold assessment, revisit after your first attempt.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Watch / current reference

Visual/source: No new video by design - integration lesson URL: <https://www.postgresql.org/docs/18/tutorial.html> Assigned scope: Reuse DE07-06/07/09 only if one design concept is still weak; do not tutorial-copy the build Current reference: PostgreSQL 18 tutorial / course-owned integrated build Reference URL: <https://www.postgresql.org/docs/18/tutorial.html> Study rule: Watch/read only the assigned idea, return to this book, type the guided work yourself, then close video/review/AI solving before the independent attempt.

Integration is where hidden modeling errors appear

A good DDL lesson can be solved in isolation. A real database must connect context, DDL, keys, transactions, dimensions, history and reconciliations without silently changing population or grain.

For the supplied source fixture, write the controls before the warehouse build:

Source control	Expected
all order-item rows	9
all line amount	2280.00
completed order-item rows	8
completed line amount	1780.00
distinct completed orders	5

Your analytical fact intentionally contains only Completed order lines, so the comparable controls are 8 rows and 1,780.00. Do not compare the fact to the unfiltered 9 / 2,280.00 source and call the difference data loss; populations must match.

Follow the build order and say why

1. Context proof - confirm database/schema first.

2. Operational/source DDL - create parent tables before child tables.
3. Known fixture - load small data whose arithmetic you can independently check.
4. Dimension design - declare business keys, surrogate keys and history rules.
5. Fact grain - write one sentence defining it before CREATE TABLE.
6. Dimension load - ensure one valid dimension version per fact event.
7. Fact load - use explicit joins and source filter.
8. Reconciliation - row count, distinct event count, amount, unmatched keys, historical attribution.
9. Changed/bad input - prove controls can fail when data is wrong.

Why two controls are never enough for every failure

Suppose one 100.00 fact row disappears and another row is duplicated with the same 100.00 amount. The grand total stays correct. A row-count/key control catches the defect.

Now imagine all 8 fact rows remain, but one 120.00 line is loaded as 210.00. The row count stays correct. An amount reconciliation catches it.

This is why "the dashboard total looks right" is weak evidence.

Validate historical groups too

Expected completed region attribution under the SCD2 customer history:

- East 660.00;
- South 600.00;
- West 520.00;
- total 1,780.00.

If the total reconciles but East/South do not, investigate temporal dimension mapping.

Build the module practical independently

Use practice/M07_CHECKPOINT_TODO.md, but choose a shipment domain with your own table names. The practical should contain:

- normalized source tables;
- named constraints;
- one transaction/rollback proof;
- a query-driven index with EXPLAIN evidence;
- fact/dimensions with an explicit fact grain;
- one SCD2 attribute;
- source/fact/reject controls;

- least-privilege plan;
- migration + backup/restore handoff plan.

Do not copy the order schema and rename order to shipment. Your data relationships and controls should genuinely change.

Architecture defense

After the build, close the book and explain for 3-5 minutes:

- what one row means in each major table;
- where business keys end and surrogate keys begin;
- why a fact measure is safe to sum;
- why the SCD join does not rewrite history;
- which database constraints protect integrity;
- which correctness properties still require pipeline validation;
- what your recovery evidence proves and what remains pending.

Changed retry

The fresh checkpoint uses receipts/suppliers and a different history attribute. If your second solution is only a mass rename of the first, it is not a fresh mastery proof.

Treat the build like a controlled release

Before each layer, define a small gate:

- source gate: expected tables load, source keys are plausible, 9 lines / 2280.00;
- population gate: Completed filter gives 8 lines / 5 orders / 1780.00;
- dimension gate: each business key maps to the intended number of versions and exactly one current row;
- fact gate: (order_id,line_no) unique, 8 rows / 1780.00;
- report gate: historical region totals reconcile to 1780.00 and C001 splits East/South at the correct date.

If the dimension gate fails, do not continue loading the fact and hope final totals expose the problem. Repair at the earliest broken boundary. That habit scales later to Airflow task boundaries, Spark stages and lakehouse quality checks.

Also save negative evidence: one deliberate invalid constraint test, one changed SCD boundary, one rejected staging row. A course portfolio that contains only screenshots of green output does not prove you can diagnose failure.

At the end, another person should be able to rebuild the model from documented source files and commands without opening your original database. That is why M07 already starts handoff

discipline before Git/Docker/cloud orchestration becomes more complex.

Evidence before you move on

End-to-end analytical database with redundant row/key/value/history controls and architecture defense. Keep the genuine first attempt/result/error. A correction made while review is open is repair evidence, not independent mastery.

Revision in your own words

Without opening the lesson, explain Build and validate a small analytical database in 60-120 seconds. State the row/object grain, one realistic failure, one database-enforced rule (if applicable), and one independent validation. Then reconstruct one small example from memory.

Protected changed retry

Fresh receipt/supplier analytical database with different keys, history attribute and reconciliation controls.

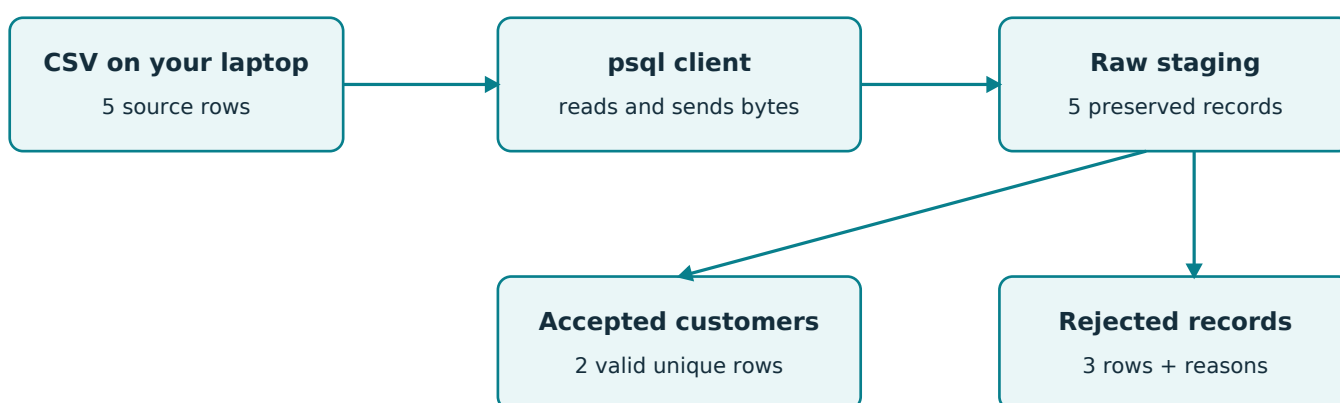
DE07-11

Bulk loading with COPY / staging boundaries

Watch first

No matching instructor video is assigned for this topic. Use the complete worked explanation and diagrams in this lesson. No generated video is required.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.



Read the picture

The CSV stays on the machine running psql. The client reads it and sends its bytes through the database connection. PostgreSQL stores the raw text in staging. Validation then separates accepted customers from rejected source rows. This is why a laptop file path works with client-side \copy but may fail with server-side COPY.

Prepare the exact command

Open a terminal in 05_M07/practice/data, where customer_stage_bad.csv exists, then start psql connected to standalone_de. Create the staging schema/table as shown in the lesson before loading. The printed \copy command is ONE physical line inside psql; a trailing shell backslash does not continue a psql meta-command. In a fresh attempt, use an empty staging table so repeated loads do not double the source count.

Watch / current reference

Visual/source: PostgreSQL 18 documentation - COPY URL:

<https://www.postgresql.org/docs/18/sql-copy.html> Assigned scope: Current server-side COPY

behavior, ON_ERROR/REJECT_LIMIT awareness and client/server distinction Current reference:

[PostgreSQL 18 - COPY Reference URL: https://www.postgresql.org/docs/18/sql-copy.html](https://www.postgresql.org/docs/18/sql-copy.html) Study

rule: Watch/read only the assigned idea, return to this book, type the guided work yourself, then close video/review/AI solving before the independent attempt.

A file crossing into PostgreSQL is a trust boundary

A CSV is text on a filesystem. A governed PostgreSQL target has typed columns, keys and constraints. Staging is the deliberate boundary between those worlds.

The safe beginner pattern is: landing file -> raw staging -> profile/validate -> accepted target + rejected/quarantine -> reconciliation.

Do not load a messy vendor file straight into final tables just because COPY is fast.

Server-side COPY versus psql \copy

This difference causes endless beginner confusion.

```
COPY m07_stage.customer_raw
FROM '/some/path/customer_stage_bad.csv'
WITH (FORMAT csv, HEADER true);
```

This SQL tells the PostgreSQL server process to read a file path from the server's point of view. The file must be accessible to that process and server-side file privileges apply.

Inside the psql program:

```
SET search_path TO m07_stage;
\copy customer_raw(customer_id,customer_name,region,signup_date_text) FROM 'customer_stage_bad.csv' CSV HEADER
```

\copy is a psql meta-command, not ordinary SQL. The client reads the local file and streams it through the connection. A laptop path can therefore work with \copy when server-side COPY

```
FROM '/laptop/path' cannot.
```

pgAdmin's import/export interface is another client workflow. Always know which machine/process is expected to open the path.

Preserve raw values first

For questionable data, make staging permissive enough to observe the original text:

```
CREATE SCHEMA IF NOT EXISTS m07_stage;
CREATE TABLE m07_stage.customer_raw (
    source_row bigint GENERATED ALWAYS AS IDENTITY,
    customer_id text,
    customer_name text,
    region text,
    signup_date_text text
);
```

If you type signup_date date in raw staging, the entire load may fail before you have captured which source value was bad. Typed promotion belongs after profiling when the source quality is uncertain.

Work the dirty five-row fixture

customer_stage_bad.csv contains 5 source rows:

- C201 valid;
- C202 blank region;

- C203 bad date text;
- duplicate C201;
- C204 valid.

For the learning contract, accepted unique valid rows = 2 and rejected/duplicate rows = 3.

Create a reject table with source_row, raw business key, raw fields and reject_reason. Do not merely delete invalid rows.

Your reconciliation is: 5 source = 2 accepted + 3 rejected.

Then validate the target has the expected two business keys and no duplicate business key.

PostgreSQL 18 awareness: COPY can handle some row errors

Current PostgreSQL 18 COPY supports options such as ON_ERROR and REJECT_LIMIT for supported input-error handling. That is useful current behavior, but it does not replace your business-quality staging design. A syntactically valid row with a blank region or duplicate business key can still be business-invalid. Keep the staging/quarantine pattern as the course's auditable method.

Common failure modes

- File path is interpreted on the wrong machine.
- Header/delimiter/encoding does not match the file.
- Direct typed load stops before raw bad values are inspectable.
- Duplicate detection is performed after a destructive dedupe, losing evidence.
- Counts are compared across different source filters.

Your independent proof

Load the dirty fixture into staging using the client/server method that actually exists on your PC. Produce accepted and rejected outputs with reasons. Save the command/tool used, source count, accepted count, rejected count and target keys.

If PostgreSQL runtime is unavailable in this environment, keep the native load evidence pending. Do not replace it with a screenshot of the SQL file.

Changed retry

Use a fresh receipt file with malformed amount, duplicate receipt-line key and one late valid row. Reconcile source = accepted + rejected and preserve every rejected source value.

Turn reject logic into inspectable SQL

After raw staging, do not immediately DELETE bad rows. Create a diagnostic view/query that assigns reasons. For example, a beginner-friendly pattern can compute flags such as blank_region, bad_date_text and duplicate_business_key, then route rows based on those flags.

A date-shaped string is not necessarily a real date: 2026-02-31 matches a simple YYYY-MM-DD pattern but is invalid. PostgreSQL 18 provides `pg_input_is_valid(text, 'date')`. Use it with a format check, then cast only in the valid CASE branch. Keep the invalid original text in quarantine. The complete example below uses this approach.

Keep duplicate handling deterministic. In the supplied fixture, the first valid C201 is accepted and the later duplicate is rejected for this exercise. A real system may use source timestamps, sequence numbers or a merge policy instead. Never use "drop duplicates, keep first" without knowing what determines first.

For each run, save a load identifier or file name when possible. Then a reject row answers not only "why rejected?" but also "from which source run?" This becomes invaluable when the same business key appears in several files.

Evidence before you move on

Raw staging -> accepted + rejected load with source=accepted+rejected reconciliation and COPY\copy explanation. Keep the genuine first attempt/result/error. A correction made while review is open is repair evidence, not independent mastery.

Revision in your own words

Without opening the lesson, explain Bulk loading with COPY / staging boundaries in 60-120 seconds. State the row/object grain, one realistic failure, one database-enforced rule (if applicable), and one independent validation. Then reconstruct one small example from memory.

Protected changed retry

Fresh dirty receipt CSV with malformed amount, duplicate line key and valid late row; preserve rejects.

A worked validation query

After the five rows are loaded, the following query assigns a single first reason to each row. The valid first C201 is kept; its later duplicate is rejected. Rank is defined by `source_row`, not arbitrary database display order.

```
WITH flagged AS
  (SELECT *,
    ROW_NUMBER() OVER (PARTITION BY customer_id
                      ORDER BY source_row) AS key_occurrence
  FROM m07_stage.customer_raw), CHECKED AS
  (SELECT *,
    CASE
      WHEN NULLIF(TRIM(region), '') IS NULL THEN 'blank_region'
      WHEN signup_date_text !~ '^d{4}-d{2}-d{2}$'
        OR NOT COALESCE(pg_input_is_valid(signup_date_text,
        'date'),
        FALSE) THEN 'invalid_date'
      WHEN key_occurrence > 1 THEN 'duplicate_customer' ELSE NULL
    END AS reject_reason
  FROM flagged)
SELECT source_row, customer_id, reject_reason,
  CASE
    WHEN reject_reason IS NULL THEN signup_date_text::date
  END AS accepted_signup_date
```

```
FROM CHECKED  
ORDER BY source_row;
```

For this supplied fixture, two rows have a null reject_reason and three have a reason. Keep the raw fields beside the reason when writing your reject table. The query is a demonstration for this fixture, not a universal customer-data contract; add explicit ID/name rules when you define a broader source contract. The official PostgreSQL 18 reference for pg_input_is_valid is <https://www.postgresql.org/docs/18/functions-info.html> .

DE07-12

Schemas, roles and least-privilege boundaries

Watch first

[1. databasetorque - PostgreSQL roles, schema and security](#)

OPTIONAL visual - no verified bounded range. Use only for role/schema/security context; PostgreSQL 18 privilege docs + learner lab are required.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Watch / current reference

Visual/source: databasetorque - PostgreSQL roles, schema and security URL:

<https://www.youtube.com/watch?v=8YSp-wTELNs> Assigned scope: Current reference:

PostgreSQL 18 - privileges and roles Reference URL:

<https://www.postgresql.org/docs/18/ddl-priv.html> Study rule: [Watch/read only the assigned](#) idea, return to this book, type the guided work yourself, then close video/review/AI solving before the independent attempt.

OPTIONAL visual - no verified bounded range; role/schema/security context only.

Least privilege is part of pipeline correctness

A pipeline that "works" only because every connection is superuser has hidden risk. Routine jobs should receive only the permissions needed for their responsibility.

PostgreSQL uses roles. A role can be allowed to log in, can own objects, can be a member of other roles, and can receive object privileges. Ownership is powerful: the owner has inherent control over the object. That is different from granting SELECT or INSERT to a routine role.

Separate responsibilities on paper first

A simple local learning design:

Role Intended responsibility

m07_owner

own schemas/tables; migration/admin use

m07_ingest

write only required staging/load targets

m07_analyst

read serving/warehouse tables only

This is not a universal production role model. It is a way to learn boundaries.

Schema privileges and table privileges are different

```
GRANT USAGE ON SCHEMA m07_dw TO m07_analyst;  
GRANT SELECT ON ALL TABLES IN SCHEMA m07_dw TO m07_analyst;
```

USAGE on a schema allows resolving/accessing objects in the namespace; it does not automatically grant SELECT on the tables. Conversely, table privileges without the necessary schema access can still leave a role unable to use the object normally.

CREATE on a schema is more powerful: it allows a role to create objects there. Do not grant it just to silence an error.

Test denials, not only successes

In a safe local instance where you control the roles, test:

- analyst can SELECT m07_dw.fact_order_line;
- analyst cannot INSERT into it;
- analyst cannot read raw staging if staging is intentionally private;
- ingest can write its approved target but cannot alter ownership/admin objects.

A denied operation is valuable evidence when denial is the policy.

You can use a separate login or, where appropriate and permitted, SET ROLE to test effective privileges. Do not experiment with role creation on a school/work/shared server.

Future objects are a common trap

A grant on all tables that exist today does not automatically mean a table created tomorrow gets the same grant. ALTER DEFAULT PRIVILEGES can define privileges for future objects created by a particular owner/role. The "particular owner" detail matters: default privileges are tied to the role whose future objects are being created.

This is why an access design should include object ownership and creation workflow, not only a list of GRANT statements.

Secrets and authentication stay outside this lesson

Database roles are authorization inside PostgreSQL. Password storage/secret managers and network authentication are broader concerns covered later. Never place real company passwords or credentials into course evidence.

Your independent proof

Create a privilege matrix before SQL. If you own a safe local PostgreSQL instance and have rights, implement the roles and save successful and denied test statements. If you cannot safely create roles, label the artifact PLAN-ONLY / NATIVE TEST PENDING. Honest pending evidence is better than a fake pass.

Changed retry

Add m07_reporter: SELECT on serving tables, no staging access and no CREATE. List the minimum schema/table grants required and one grant you deliberately refuse.

Read one privilege failure like an engineer

Suppose m07_analyst can resolve m07_dw.fact_order_line and SELECT from it, but an INSERT fails with permission denied for table fact_order_line. That error is a pass if the policy says analysts are read-only.

Do not respond by granting ALL PRIVILEGES. Find the operation the role genuinely needs and grant only that scope. Similarly, granting table SELECT but forgetting schema USAGE can produce a different access failure; fix the missing layer rather than overprivileging the whole database.

Ownership should also be deliberate. If your routine ingestion login owns every table, it can often do much more than its job requires. Many production setups separate migration/object ownership from runtime roles. M07's small m07_owner / m07_ingest / m07_analyst model is meant to make that separation visible.

When you document default privileges, state whose future objects the rule applies to. A common surprise is configuring defaults as one role and later creating tables as another role, then wondering why the expected grants did not appear.

A strong handoff therefore includes both the privilege matrix and the object-creation/ownership convention.

Evidence before you move on

Least-privilege matrix plus real success/denial tests if safe, otherwise explicitly plan-only. Keep the genuine first attempt/result/error. A correction made while review is open is repair evidence, not independent mastery.

Revision in your own words

Without opening the lesson, explain Schemas, roles and least-privilege boundaries in 60-120 seconds. State the row/object grain, one realistic failure, one database-enforced rule (if applicable), and one independent validation. Then reconstruct one small example from memory.

Protected changed retry

Reporter role can read serving only; cannot see staging/write/create. List exact minimum grants.

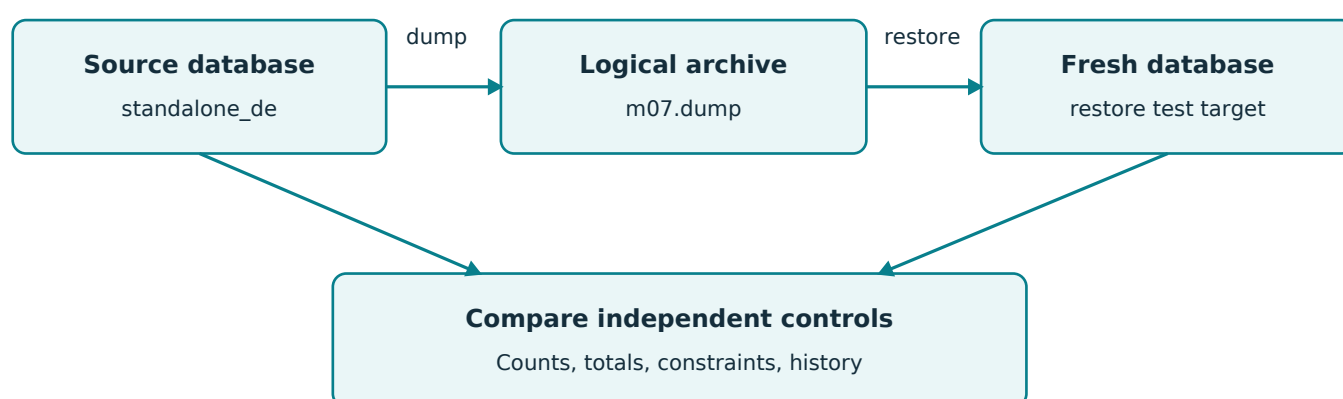
DE07-13

Schema change, backup/restore and handoff validation

Watch first

No matching instructor video is assigned for this topic. Use the complete worked explanation and diagrams in this lesson. No generated video is required.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.



Picture the recovery test

The first database remains your source. `pg_dump` writes a portable logical archive. `pg_restore` reconstructs a different, empty database from that archive. You then run the same counts, totals, keys and history query on both. The check proves reconstruction; merely opening the archive file proves very little.

Work through the commands

Run the three printed commands in your operating-system terminal, one at a time. First confirm PostgreSQL command-line tools are available and your connection reaches `standalone_de`. The new target `standalone_de_restore_test` must be disposable and must not already contain your working tables. If `createdb` says it already exists, choose a new test database name and use that same name in `pg_restore`; do not delete your working database to clear the error. A password prompt may be normal for your installation. Keep passwords out of the command text. After restoration, run the supplied control SQL against the restore target and compare with the source: 9 source rows / 2280.00 and 8 completed facts / 1780.00. A mismatch means the recovery task is not complete even if `pg_restore` exited successfully.

Watch / current reference

Visual/source: PostgreSQL 18 documentation - pg_dump URL:

<https://www.postgresql.org/docs/18/app-pgdump.html> Assigned scope: Logical dump formats

and recovery boundary; course requires restore validation Current reference: PostgreSQL 18 -

[pg_dump Reference URL: https://www.postgresql.org/docs/18/app-pgdump.html](https://www.postgresql.org/docs/18/app-pgdump.html) Study rule:

Watch/read only the assigned idea, return to this book, type the guided work yourself, then close video/review/AI solving before the independent attempt.

A database handoff is proven by recovery

A SQL file in Git is useful. A .dump file is useful. Neither proves that another clean target can reconstruct the database and pass the same controls.

Treat schema change and recovery as two connected engineering questions:

1. Can I evolve this database without leaving consumers in an unknown state?
2. Can I rebuild/recover it somewhere else and prove the rebuilt copy is correct?

Use staged schema changes

Suppose a production-like table needs a new required source_system column. A beginner's instinct is:

```
ALTER TABLE big_table  
ADD COLUMN source_system text NOT NULL;
```

On a populated table that may fail immediately because existing rows have no value, and some changes can hold locks or affect old application versions.

A safer conceptual sequence is:

1. add the column nullable or with a deliberately safe transition rule;
2. backfill existing rows in controlled batches/transaction strategy;
3. validate there are no remaining invalid/missing values;
4. update writers/readers;
5. enforce NOT NULL only when the contract is true;
6. keep rollback/repair thinking documented.

The exact production migration strategy depends on size, traffic, locks and application compatibility. M07 teaches the habit, not universal zero-downtime migration magic.

Understand what pg_dump does

pg_dump makes a logical export of one PostgreSQL database. It can produce plain SQL or archive formats. It does not, by itself, export cluster-wide roles/tablespaces; those global objects require separate handling such as pg_dumpall options/administration.

Example local commands for a custom-format dump:

```
pg_dump -Fc -d standalone_de -f m07.dump  
createdb standalone_de_restore_test
```

```
pg_restore -d standalone_de_restore_test m07.dump
```

These are shell commands, not SQL. Connection parameters and authentication vary. Do not paste passwords into a shared shell history or course file.

The restore target must be different

Restoring back over the same working database does not prove you can reconstruct a clean environment. Use a separate database such as `standalone_de_restore_test` (or another isolated test target you control).

After restore, rerun independent controls:

- expected schemas/tables exist;
- source rows = 9;
- source amount = 2,280.00;
- completed fact rows = 8;
- completed fact amount = 1,780.00;
- C001 historical orders map East then South;
- representative constraints/keys still exist;
- one representative query returns the same business result.

A file-exists check is not a recovery test.

Handoff is more than backup

A second engineer needs:

- ordered setup/migration SQL;
- required PostgreSQL version/extension assumptions;
- load order and source-file contracts;
- role/privilege plan;
- known reconciliation controls;
- backup/restore commands;
- what was actually tested versus still pending;
- which folders are source artifacts versus generated evidence.

Hidden local paths are a common failure. If your restore instructions say `/Users/ashish/Desktop/final2/...`, the handoff is not reproducible on another machine.

Your independent proof

Make one additive schema change in the learner DB. Document pre-check, forward steps, validation and repair/rollback thinking. Create a logical dump and restore it to a second database if the PostgreSQL command-line tools are available. Run the controls again on the restore target.

If the tools are unavailable, leave the restore item pending. Do not convert a written plan into "PASS."

Changed retry

Use the fresh receipt/supplier model. Restore it to a newly named database and ask: Could another person reproduce the schema and verify the core counts without my original working directory? Repair every hidden dependency the answer exposes.

Restore validation should detect more than missing rows

After restoring to a clean target, check several layers:

1. schemas/tables/views/indexes expected by the module exist;
2. key constraints still reject a known invalid row in a rolled-back test;
3. row and amount controls match;
4. the SCD2 as-of report still maps O101 East and O102 South;
5. a representative role/privilege plan can be recreated (remember a single-database `pg_dump` does not automatically capture every cluster-global role);
6. the README commands do not depend on your personal absolute path.

Why test constraints after restore? A data-only export can reproduce rows while omitting some schema objects depending on how the backup was made/restored. Why test the business report? Object counts can all match while a migration altered logic or history. Recovery evidence should climb from structure to data to business result.

For a schema migration, capture before/after metadata as well as data. If you add `source_system`, prove pre-existing rows were backfilled to the intended value before enforcing NOT NULL. If an application/pipeline version must coexist during rollout, document that compatibility assumption instead of pretending the database change is isolated.

This is the beginning of DataOps thinking that will become much more explicit in M16. A trustworthy handoff lets another engineer reproduce the database without inheriting invisible assumptions from your laptop, shell history or memory.

Evidence before you move on

Staged schema change + `pg_dump/restore-to-clean-target` evidence if native tools exist; honest pending status otherwise. Keep the genuine first attempt/result/error. A correction made while review is open is repair evidence, not independent mastery.

Revision in your own words

Without opening the lesson, explain Schema change, backup/restore and handoff validation in 60-120 seconds. State the row/object grain, one realistic failure, one database-enforced rule (if applicable), and one independent validation. Then reconstruct one small example from memory.

Protected changed retry

Fresh model dump/restore into a new target; eliminate one hidden absolute-path or local-state dependency.